

h1. What is dynamic typing in Python?

Dynamic typing is one of Python's core features. In Python, variables are not bound to a specific type at declaration. Instead, the type is determined at runtime based on the assigned value. This means that a single variable can hold data of different types throughout its lifetime hence making Python a flexible and easy-to-use language for rapid development and prototyping. Dynamic typing means that you do not need to declare the data type of a variable explicitly in Python.

2. What is the difference between mutable and immutable data types?

Immutable Objects are of in-built datatypes like int, float, bool, string, Unicode, and tuple. In simple words, an immutable object can't be changed after it is created.

Mutable objects can be changed after they are created. Eg: list, dict, set.

3. What are local and global variables?

Local Variables

- Local variables are created inside a function or a block.
- They can be used only within that function or block.
- They are deleted after the function finishes executing.
- The same local variable name can be used in different functions without conflict.
- They are used for temporary data needed only in a specific task.

```
def add_numbers():  
    x = 10      # local variable  
    y = 20      # local variable  
    print(x + y)  
  
add_numbers()  
# print(x)  ❌ Error: x is not accessible outside the function
```

Global Variables

- Global variables are created outside all functions.
- They can be accessed anywhere in the program.
- They exist throughout the program execution.
- Using too many global variables can make programs harder to understand and debug.
- They are used when multiple parts of a program need the same data.

```
> ✓  
total = 100    # global variable  
  
def show_total():  
    print(total)  
  
show_total()  
print(total)
```

4. What is the purpose of the return statement in a function?

The return statement is used inside a function to send a value back to the place where the function was called. Once return is executed, the

function stops running, and any code written after it is ignored. If no value is returned, Python automatically returns None.

```
def square(n):  
    return n * n  
  
res = square(4)  
print(res)
```

5. What is list comprehension?

List comprehension is a short and easy way to create a list in Python using a single line of code instead of a loop.

```
Without List Comprehension  
  
squares = []  
for i in range(5):  
    squares.append(i*i)  
  
With List Comprehension  
  
squares = [i*i for i in range(5)]
```

6. Difference between break, continue, and pass

Loops in Python are used to repeat tasks automatically.

Sometimes, we may want to:

- stop the loop completely
- skip one iteration
- do nothing for a condition

For these purposes, Python provides loop control statements.

They change the normal flow of a loop.

1. continue Statement

- Used to skip the current iteration of the loop.
- Control goes back to the start of the loop for the next iteration.

2. break Statement

- Used to exit the loop completely.
- The loop stops immediately when break is executed.

3. pass Statement

- Does nothing.
- Used as a placeholder when a statement is required but no action is needed.
- Commonly used in empty loops, functions, or classes.

CONTINUE

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

BREAK

```
for i in range(5):  
    if i == 3:  
        break  
    print(i)
```

PASS

```
for i in range(5):  
    if i == 3:  
        pass  
    print(i)
```

7. What is recursion? Give a simple example.

Recursion is when a function calls itself to solve a smaller part of a problem until a base condition is met. Must have a base case to stop, otherwise it will run forever.

```
def factorial(n):  
    if n == 0:          # base case  
        return 1  
    else:  
        return n * factorial(n-1)  # recursive call  
  
print(factorial(5))    # Output: 120
```

8. What is a lambda function?

Lambda Functions are anonymous functions means that the function is without a name. As we already know `def` keyword is used to define a normal function in Python. Similarly, `lambda` keyword is used to define an anonymous function in python. It is often used for **short, simple operations** without defining a full function using `def`.

```
add = lambda x, y: x + y  
print(add(5, 3))    # Output: 8
```

9. What is the difference between `is` and `==`?

1. `==` (Equality Operator)

- Checks if the values of two objects are equal.
- Compares contents, not memory location.

2. `is` (Identity Operator)

- Checks if two objects are actually the same object in memory.
- Compares memory addresses, not values.

```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b) # True → values are equal
print(a is b) # False → different objects in memory
```

10. What are docstrings and why are they important?

Docstrings (Documentation Strings) are special strings used to document Python code. They provide a description of what a module, class, function or method does. It describe what it does.

Docstrings are Important because:

1. **Documentation:** Explains what a function, class, or module does.
2. **Readability:** Makes code easier to understand for others (and for you in the future).
3. **Help Functionality:** Python's built-in `help()` function uses docstrings.

```
def greet(name):
    """This function greets the user by their name."""
    return f"Hello, {name}!"
```